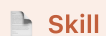


Skill · Tool MCP & Subagentes

Os quatro pilares dos sistemas de agentes de IA modernos — o que cada um é, quando usar e como se combinam.



Skill



Tool



MCP



Subagente



Conheça cada *peça*

Clique nas abas para ver o que é, como funciona e um exemplo.



Skill



Tool



MCP



Subagente

O playbook do agente

Arquivo Markdown (SKILL.md) com instruções que ensinam o agente a seguir um fluxo. Injetado no contexto quando relevante — **não executa código**, guia o raciocínio.

Ex.: Skill "code-review" checa SQL injection, XSS e auth sempre nessa ordem.

Executa código?

Não

Quem age?

O próprio modelo, guiado pelo texto

Em uma linha

Define **como** o agente deve pensar/agir

Executa código?

Sim

Quem age?

Sistema externo; resultado volta ao modelo

Em uma linha

Define **o que** o agente pode executar

Capacidade de execução real

Função com schema (nome, descrição, parâmetros) que o modelo pode chamar. A **execução acontece fora do LLM** — o modelo decide quando chamar, envia parâmetros e recebe o resultado.

Ex.: web_search, bash, read_file, send_email.

Tabela *comparativa*

CONCEITO	O QUE É	EXECUTA CÓDIGO?	QUEM AGE	EM UMA LINHA
 Skill	Playbook em Markdown	Não	O próprio modelo	Como pensar/agir
 Tool	Função com schema	Sim	Sistema externo	O que pode executar
 MCP	Protocolo padrão	Infra	Servidor externo	Como conectar tools
 Subagente	LLM para subtarefa	Sim (via tools)	Outro LLM	Quem trabalha em paralelo



Na prática eles se complementam: uma Skill pode instruir o uso de Tools via MCP, delegando partes a Subagentes.

Os 4 pilares *juntos*

Pedido: "Pesquise sobre IA generativa e crie um relatório revisado."

1

Skill ativa

A skill `research-report` é injetada com o fluxo de trabalho.

2

Tools executam

O agente chama `web_search` e `read_file`.

3

MCP conecta

Google Docs e Slack expostos via servidores MCP.

4

Subagente revisa

Um especialista recebe o rascunho e devolve a versão final.



A **Skill** orchestra o raciocínio · as **Tools** executam ações · o **MCP** padroniza conexões · os **Subagentes** dividem o trabalho.

Quando usar *cada pilar*?

Expanda cada pergunta.

Precisa padronizar o raciocínio do agente? → Skill



Quando o agente deve seguir um fluxo, formato ou checklist específico em toda execução.

O agente precisa agir no mundo real? → Tool



Quando precisa ler arquivos, chamar APIs, enviar e-mails ou executar comandos.

Quer integrar com serviços de forma padrão? → MCP



Quando há um servidor MCP disponível (Gmail, Slack, GitHub) e você quer plug-and-play.

A tarefa é complexa demais para um só agente? → Subagente



Quando convém dividir em subtarefas autônomas com raciocínio independente.

💡 Lembre-se: os pilares não se excluem — em projetos reais, você usa os quatro juntos

Teste *rápido*

Você quer que o agente possa enviar e-mails de verdade pelo Gmail, sem escrever código de integração. Que pilares combinam melhor?

A Só uma Skill

B Uma Tool exposta via MCP

C Só um Subagente

D Só um Subagente com Skill

Resposta: B — Tool via MCP. Enviar e-mail é **agir no mundo real** → precisa de uma **Tool** (`send_email`). Fazer isso **sem código de integração** é exatamente o papel do **MCP**: o servidor MCP do Gmail já expõe essa tool de forma padronizada. Uma Skill sozinha só instrui, não executa.



Quatro peças, *um sistema*



Skill

Como pensar (Markdown)



Tool

O que executar (função)



MCP

Como conectar (protocolo)



Subagente

Quem paraleliza (LLM)

Fim do módulo **3 • Agent Skills** — do "por que" à arquitetura completa dos agentes.